**PROJECT REPORT**

**TITLE: NOTES MANAGEMENT SYSTEM (JAVA)**

**Submitted by:** SURAJ GUPTA

**Roll Number:** 24BAI10780

**SLOT:F11+F12**

**Subject:** Object-Oriented Programming (Java)

**Date:** March 31, 2026

---

## 1. ABSTRACT

The **Notes Management System** is a console-based Java application designed to provide a digital platform for creating, storing, and organizing personal information. The system leverages file-handling techniques to ensure data persistence, allowing users to maintain a long-term record of their notes without the need for an external database. This project serves as a practical implementation of Core Java concepts including Encapsulation, File I/O, and the Collections Framework.

---

## 2. INTRODUCTION

In the modern digital era, organizing thoughts and tasks efficiently is crucial. This project, the **Notes Management System**, offers a lightweight alternative to heavy note-taking software. Written entirely in Java, it provides a menu-driven interface where users can perform CRUD (Create, Read, Update, Delete) operations on their notes.

The primary focus of this project is to demonstrate how a modular application can be built by separating concerns into different classes, ensuring the code is maintainable and scalable.

---

## 3. SYSTEM REQUIREMENTS SPECIFICATION

### 3.1 Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Programming Language:** Java JDK 17 or higher.
- **Development Environment:** IntelliJ IDEA, Eclipse, or NetBeans.
- **Storage:** Local text file (.txt) for data persistence.

### 3.2 Hardware Requirements:

- **Processor:** Minimum 1 GHz.
- **RAM:** 512 MB or higher.
- **Hard Disk Space:** 10 MB (for application and data storage).

---

## 4. SYSTEM DESIGN AND ARCHITECTURE

### 4.1 Modular Structure

The project is divided into four primary modules:

1. **UI Module (Main.java):** Handles user interactions and menu navigation.
2. **Logic Module (NoteManager.java):** Contains algorithms for searching, filtering, and managing the list of notes.
3. **Data Model (Note.java):** Defines the structure and properties of a "Note" object.
4. **Storage Module (Storage.java):** Manages file reading and writing operations.

### 4.2 Data Flow

When the user adds a note:

1. Input is collected via the Main class.
2. A Note object is instantiated in NoteManager.
3. The NoteManager sends the updated list to Storage.
4. Storage serializes the data into a pipe-delimited (|||) format and writes it to notes.txt.

---

## 5. IMPLEMENTATION DETAILS

### 5.1 Object-Oriented Principles Used:

- **Encapsulation:** Used in the Note class to protect data fields (ID, Title, Content) using private access modifiers and public getters/setters.
- **Inheritance & Polymorphism:** Utilized through the use of Java Collections (ArrayList) and overriding the toString() method.
- **Abstraction:** The user interacts with high-level menu options while the complex file-parsing logic remains hidden within the Storage class.

### 5.2 Key Algorithms:

- **Auto-Increment ID:** The system scans the existing list of notes to find the highest ID and assigns $ID_{new} = ID_{max} + 1$.
- **Keyword Search:** Implements a case-insensitive search algorithm that iterates through the ArrayList to find matches in titles or content.
- **Timestamping:** Uses the java.time.LocalDateTime class to automatically log the creation time of each note.

---

## 6. USER INTERFACE (MENU STRUCTURE)

The application provides the following options to the user:

1. **Add Note:** Input title, category, and multi-line content.
2. **List All Notes:** Displays a summary table of all stored notes.
3. **View Note:** Show full details of a specific note by ID.
4. **Edit Note:** Modify existing note details.
5. **Delete Note:** Permanently remove a note from storage.
6. **Search Notes:** Find notes containing specific keywords.
7. **Filter by Category:** View notes belonging to a specific group (e.g., Work, Personal).
8. **Exit:** Securely save data and close the application.

---

## 7. TESTING AND RESULTS

The system was tested with the following scenarios:

- **Data Persistence Test:** Saved 10 notes, closed the program, and restarted. All 10 notes were successfully reloaded from the file.
- **Empty Search Test:** Searching for a non-existent keyword correctly returned a "No matches found" message.
- **Boundary Test:** Attempting to delete a note with an invalid ID (e.g., ID 999 when only 5 notes exist) was handled without a system crash.

---

## 8. ADVANTAGES AND LIMITATIONS

### 8.1 Advantages:

- **Zero Dependency:** Does not require a heavy database engine like MySQL or Oracle.
- **Portability:** The data file can be moved across systems easily.
- **Readable Format:** The storage file is human-readable, allowing for manual backups.

### 8.2 Limitations:

- **No GUI:** Operates only in the terminal/console.
- **Concurrent Access:** Does not support multiple users editing the file at the same time.
- **Security:** Does not include password protection or encryption for sensitive notes.

---

## 9. CONCLUSION

The **Notes Management System** successfully achieves its objective of providing a reliable, persistent note-taking tool. Through this project, I have gained hands-on experience in managing the application lifecycle—from design and implementation to testing and debugging. The modular nature

of the code ensures that future enhancements, such as a Graphical User Interface (GUI) or database integration, can be added with minimal changes to the existing logic.

---

**10. BIBLIOGRAPHY**


- Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.
- Oracle Java Documentation: https://docs.oracle.com/en/java/
- W3Schools Java Tutorial for File Handling and Collections.